Lecture 17: Recurrences, Master Theorem, and Runtime Models

Topics

- Writing recurrence relations for recursive algorithms
- Common recurrence patterns in divide-and-conquer
- The Master Theorem (simplified version)
- Applying the Master Theorem to mergesort, binary search, etc.
- Python runtime models and hidden costs

Goals

- Derive recurrences from code structure
- Recognize the three Master Theorem cases
- Determine time complexity using the Master Theorem
- Understand when recursion overhead matters in Python

Note: This is a theory lecture — slide-heavy with code for clarification only.

Roadmap

First half (≈45 min)

- What are recurrences? From code to math
- Base cases and recursive cases
- Tree method: visualizing recursive calls
- Common patterns: $T(n) = aT(n/b) + f(n)$
- In-class exercise 1: Multiple choice (commit required)

Break (3 min)

Second half (≈45 min)

- The Master Theorem: three cases
- Applying the Master Theorem to classic algorithms
- When the Master Theorem doesn't apply
- Python runtime models: recursion depth and overhead
- In-class exercise 2: Short answer (commit required)
- Complexity checkpoints and wrap-up

Part 1: What Are Recurrences?

Definition

A **recurrence relation** expresses the runtime of an algorithm in terms of the runtime on smaller inputs.

General form:

```
T(n) = work on smaller problems + work at this level
```

Example (factorial):

```python
def factorial(n):
    if n <= 1:              # Base case: O(1)
        return 1
    return n * factorial(n - 1)  # T(n-1) + O(1)
```

Recurrence:

```
T(n) = T(n - 1) + O(1)
T(1) = O(1)
```

Solution: T(n) = O(n)

Example: Binary search

```python
def binary_search(arr, target, left, right):
    if left > right:          # Base case: O(1)
        return -1

    mid = (left + right) // 2   # O(1)

    if arr[mid] == target:      # O(1)
        return mid
    elif arr[mid] < target:
        return binary_search(arr, target, mid + 1, right)  # T(n/2)
    else:
        return binary_search(arr, target, left, mid - 1)   # T(n/2)
```

Recurrence:

```
T(n) = T(n/2) + O(1)
T(1) = O(1)
```

What's the solution? We'll use the Master Theorem to find out!

Example: Mergesort

```python
def mergesort(arr):
    if len(arr) <= 1:           # Base case: O(1)
        return arr

    mid = len(arr) // 2         # O(1)
    left = mergesort(arr[:mid])     # T(n/2)
    right = mergesort(arr[mid:])    # T(n/2)

    return merge(left, right)  # O(n)
```

Recurrence:

```
T(n) = 2T(n/2) + O(n)
T(1) = O(1)
```

Analysis:

- Split into 2 subproblems of size n/2
- Merge takes O(n) to combine results
- Solution: T(n) = O(n log n)

Key Definitions

Recurrence relation components

Base case: T(1) or T(0) — what happens when the problem is trivial?

Recursive case: T(n) = ... — how does the problem break down?

Parameters:

- `a` = number of recursive calls
- `b` = factor by which problem size shrinks
- `f(n)` = work done at this level (excluding recursive calls)

Standard form

```
T(n) = a·T(n/b) + f(n)
```

Examples:

- Binary search: a=1, b=2, f(n)=O(1)
- Mergesort: a=2, b=2, f(n)=O(n)
- Naive matrix multiply: a=8, b=2, f(n)=O(1)

Part 2: The Tree Method

Visualizing recursion as a tree

Example: T(n) = 2T(n/2) + n (mergesort)

```
Level 0:          n                       Work: n
                 / \
Level 1:      n/2   n/2                   Work: n/2 + n/2 = n
              / \   / \
Level 2:   n/4 n/4 n/4 n/4                Work: 4·(n/4) = n
           ...
Level log n: n leaves of size 1           Work: n·1 = n
```

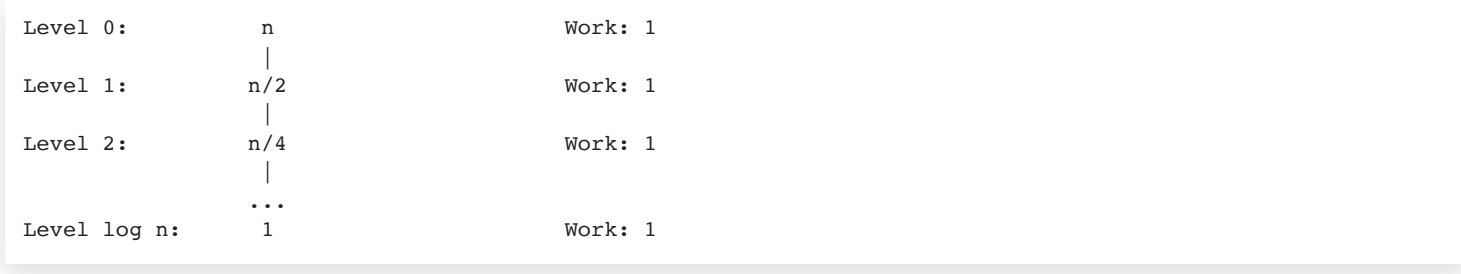
Total work:

- Height of tree: $\log_2 n$ levels
- Work per level: n
- Total: n log n

Therefore: T(n) = O(n log n)

Tree method: Binary search

Recurrence: $T(n) = T(n/2) + 1$



Total work:

- Height: $\log_2 n$
- Work per level: 1
- Total: log n

Therefore: $T(n) = O(\log n)$

Common patterns

Recurrence	Tree height	Work per level	Total
T(n) = T(n-1) + O(1)	n	O(1)	**O(n)**
T(n) = T(n/2) + O(1)	log n	O(1)	**O(log n)**
T(n) = 2T(n/2) + O(n)	log n	O(n)	**O(n log n)**
T(n) = 2T(n/2) + O(1)	log n	O(2^k)	**O(n)**
T(n) = T(n-1) + O(n)	n	O(k·n)	**O(n^2)**

Pattern recognition:

- Linear recursion (n → n-1): usually O(n)
- Binary recursion (n → n/2): usually involves log n
- Work per level determines constant factor or extra log n

Mental models

The ”shrinking stack” model

Linear recursion (n → n-1):

- Call stack depth: n
- Work per call: O(1)
- Total: O(n)

The ”divide-and-conquer” model

Binary recursion (n → n/2):

- Call tree depth: log n
- Number of calls at depth k: 2^k
- Work at depth k: depends on f(n)

The ”branching factor” intuition

High branching (a >> b):

- Number of leaves dominates (exponential growth)
- Example: $T(n) = 4T(n/2) + O(n) \rightarrow O(n^2)$

Low branching (a << b):

- Root work dominates (geometric decay)
- Example: T(n) = T(n/2) + O(n) → O(n)

Exercise 1: Multiple Choice (15 min)

Instructions: Answer the following questions. Write your answers in a Python comment block in `exercise1.py`, then commit.

```
"""
Lecture 17 Exercise 1: Recurrence Relations

1. What is the recurrence for this code?
   def mystery(n):
       if n <= 1:
           return 1
       return mystery(n // 3) + mystery(n // 3) + n

   A) T(n) = T(n/3) + O(n)
   B) T(n) = 2T(n/3) + O(n)
   C) T(n) = 3T(n/3) + O(n)
   D) T(n) = 2T(n/2) + O(n)

   Answer:

2. Using the tree method, T(n) = 2T(n/2) + O(1) has time complexity:

   A) O(log n)
   B) O(n)
   C) O(n log n)
   D) O(n²)
```

```
   Answer:

3. The recurrence T(n) = T(n-1) + O(n) solves to:

   A) O(n)
   B) O(n log n)
   C) O(n²)
   D) O(2^n)

   Answer:

4. Which recurrence pattern appears in binary search?

   A) T(n) = T(n-1) + O(1)
   B) T(n) = T(n/2) + O(1)
   C) T(n) = 2T(n/2) + O(1)
   D) T(n) = 2T(n/2) + O(n)

   Answer:
"""
```

Commit: `git add exercise1.py && git commit -m "Lecture 17 Exercise 1: Recurrence MC"`

Exercise 1 Solutions

1. Answer: B — Two recursive calls to n/3, plus O(n) work

- $T(n) = 2T(n/3) + O(n)$

2. Answer: B — $T(n) = 2T(n/2) + O(1)$

- Height: log n
- Level k has 2^k nodes, each doing O(1) work → 2^k total
- Sum geometric series: $1 + 2 + 4 + \ldots + 2^{(\log n)} = 2^{(\log n + 1)} - 1 = 2n - 1 = O(n)$

3. Answer: C — Linear recursion with linear work

- $T(n) = T(n-1) + n = n + (n-1) + (n-2) + \ldots + 1 = n(n+1)/2 = O(n^2)$

4. Answer: B — Binary search splits in half, does O(1) work

- $T(n) = T(n/2) + O(1) \rightarrow O(\log n)$

Break (3 minutes)

Stand up, stretch. Next: The Master Theorem and applying it to classic algorithms.

Part 3: The Master Theorem

Simplified statement

For recurrences of the form:

```
T(n) = a·T(n/b) + f(n)
```

Where:

- a ≥ 1 (number of subproblems)
- b > 1 (shrinkage factor)
- f(n) = work at this level (asymptotically positive)

Compare f(n) to n^(log_b a):

Case 1: f(n) = O(n^c) where c < log_b a

- **Leaves dominate** → T(n) = Θ(n^(log_b a))

Case 2: $f(n) = \Theta(n^{\log_b a})$

- **Balanced** → $T(n) = \Theta(n^{\log_b a} \cdot \log n)$

Case 3: $f(n) = \Omega(n^c)$ where $c > \log_b a$ (+ regularity)

- **Root dominates** → $T(n) = \Theta(f(n))$

Master Theorem: Intuition

Critical exponent: log_b a

- This is the "natural" growth rate of the recursion tree
- Number of leaves = a^(log_b n) = n^(log_b a)

Three cases:

Case 1: Leaves dominate (f(n) grows slower)

- Most work happens at the bottom of the tree
- Example: T(n) = 4T(n/2) + O(n)
- log_b a = $\log_2 4 = 2$, so n^2 leaves dominate O(n) per-level work
- Result: T(n) = $\Theta(n^2)$

Case 2: Balanced (f(n) grows at same rate)

- Work is evenly distributed across all levels
- Example: T(n) = 2T(n/2) + O(n)
- log_b a = $\log_2 2 = 1$, balanced with f(n) = n
- Result: T(n) = $\Theta(n \log n)$

Case 3: Root dominates (f(n) grows faster)

- Most work happens at the top of the tree
- Example: $T(n) = T(n/2) + O(n)$
- $\log_b a = \log_2 1 = 0$, root work $O(n)$ dominates
- Result: $T(n) = \Theta(n)$

Example: Binary search

Recurrence: $T(n) = T(n/2) + O(1)$

Parameters:

- $a = 1$ (one recursive call)
- $b = 2$ (divide by 2)
- $f(n) = O(1)$

Critical exponent: $\log_b a = \log_2 1 = 0$

Compare:

- $n^{(\log_b a)} = n^0 = 1$
- $f(n) = O(1) = \Theta(n^0)$

Case 2 applies! (balanced)

Result: $T(n) = \Theta(n^0 \cdot \log n) = \mathbf{\Theta(\log n)}$

Example: Mergesort

Recurrence: $T(n) = 2T(n/2) + O(n)$

Parameters:

- $a = 2$ (two recursive calls)
- $b = 2$ (divide by 2)
- $f(n) = O(n)$

Critical exponent: $\log_b a = \log_2 2 = 1$

Compare:

- $n^{(\log_b a)} = n^1 = n$
- $f(n) = \Theta(n)$

Case 2 applies! (balanced)

Result: $T(n) = \Theta(n^1 \cdot \log n) = \mathbf{\Theta(n \log n)}$

Example: Naive matrix multiplication

Recurrence: $T(n) = 8T(n/2) + O(n^2)$

Context: Divide n×n matrix into 4 quadrants, recursively multiply 8 pairs, combine in $O(n^2)$

Parameters:

- a = 8 (eight recursive calls)
- b = 2 (divide by 2)
- $f(n) = O(n^2)$

Critical exponent: $\log_b a = \log_2 8 = 3$

Compare:

- n^(log_b a) = n^3
- $f(n) = O(n^2) = O(n^c)$ where $c = 2 < 3$

Case 1 applies! (leaves dominate)

Result: $T(n) = \Theta(n^3) =$ **$\Theta(n^3)$**

Example: Finding maximum

Recurrence: $T(n) = T(n/2) + O(n)$

Context: Split array, recursively find max of half, scan other half

Parameters:

- $a = 1$ (one recursive call)
- $b = 2$ (divide by 2)
- $f(n) = O(n)$

Critical exponent: $\log_b a = \log_2 1 = 0$

Compare:

- $n^{(\log_b a)} = n^0 = 1$
- $f(n) = O(n) = \Omega(n^c)$ where $c = 1 > 0$

Case 3 applies! (root dominates)

Result: $T(n) = \Theta(f(n)) =$ **$\Theta(n)$**

Master Theorem summary table

Algorithm	Recurrence	a	b	log_b a	f(n)	Case	Result
Binary search	$T(n) = T(n/2) + 1$	1	2	0	$\Theta(1)$	2	**$O(\log n)$**
Mergesort	$T(n) = 2T(n/2) + n$	2	2	1	$\Theta(n)$	2	**$O(n \log n)$**
Naive matmul	$T(n) = 8T(n/2) + n^2$	8	2	3	$\Theta(n^2)$	1	**$O(n^3)$**
Recursive max	$T(n) = T(n/2) + n$	1	2	0	$\Theta(n)$	3	**$O(n)$**
Tree traversal	$T(n) = 2T(n/2) + 1$	2	2	1	$\Theta(1)$	1	**$O(n)$**
Strassen matmul	$T(n) = 7T(n/2) + n^2$	7	2	~2.81	$\Theta(n^2)$	1	**$O(n^{2.81})$**

When the Master Theorem doesn't apply

Non-standard form

Example 1: $T(n) = 2T(n/2) + n \log n$

- Not quite case 2 (has extra log factor)
- Actual solution: $\Theta(n \log^2 n)$
- Use recursion tree method instead

Example 2: $T(n) = T(n/3) + T(2n/3) + n$

- Unequal split sizes
- Master Theorem requires equal splits
- Use substitution method or tree analysis

Example 3: $T(n) = T(n - 1) + n$

- Not dividing by constant factor
- This is linear recursion, not divide-and-conquer
- Direct summation: $T(n) = n + (n-1) + \ldots + 1 = O(n^2)$

Regularity condition fails (Case 3)

Case 3 requires: $a \cdot f(n/b) \leq c \cdot f(n)$ for some $c < 1$

- Ensures geometric decay in tree
- Most polynomial f(n) satisfy this
- Pathological cases exist but rare in practice

Part 4: Python Runtime Models

Recursion overhead in Python

Call stack costs:

- Each function call: new stack frame (~200-500 bytes)
- Parameter passing: references (cheap) or copies (expensive)
- Return value: reference (cheap)

Recursion depth limit:

- Default: ~1000 calls (sys.getrecursionlimit())
- Can increase with sys.setrecursionlimit()
- Stack overflow if exceeded

When recursion hurts:

- Deep recursion (n > 1000): risk stack overflow
- Linear recursion with large constants: iteration is faster
- Python has no tail-call optimization

Example: Recursive vs iterative

Recursive factorial (risky for large n):

```python
def factorial_recursive(n):
    if n <= 1:
        return 1
    return n * factorial_recursive(n - 1)
# Stack depth: O(n)
# Fails for n > ~1000
```

Iterative factorial (safe):

```python
def factorial_iterative(n):
    result = 1
    for i in range(2, n + 1):
        result *= i
    return result
# Stack depth: O(1)
# Works for any n (until integer overflow)
```

Both are O(n) time, but iterative is faster and safer in Python.

Hidden costs in divide-and-conquer

Array slicing creates copies:

```python
def mergesort_bad(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = mergesort_bad(arr[:mid])      # O(n) copy!
    right = mergesort_bad(arr[mid:])     # O(n) copy!
    return merge(left, right)
```

Problem: Each level does O(n) copying, log n levels → O(n log n) **extra** space and time.

Solution: Use indices instead of slices

```python
def mergesort_good(arr, left, right):
    if left >= right:
        return
    mid = (left + right) // 2
    mergesort_good(arr, left, mid)       # No copy
    mergesort_good(arr, mid + 1, right)  # No copy
    merge_inplace(arr, left, mid, right)
```

Same asymptotic complexity, but 2-3x faster in practice.

Common pitfalls

1. Forgetting base cases

```python
def bad_factorial(n):
    return n * bad_factorial(n - 1)  # Infinite recursion!
```

Fix: Always check for base case first.

2. Miscounting recursive calls

```python
def mystery(n):
    if n <= 1:
        return 1
    x = mystery(n // 2)  # Only ONE call
    return x + x         # NOT two calls!
```

Recurrence: $T(n) = T(n/2) + O(1)$, not $T(n) = 2T(n/2) + O(1)$

3. Ignoring hidden costs

```python
def concat_recursive(arr):
    if len(arr) <= 1:
        return arr
```

```
    return arr[:len(arr)//2] + concat_recursive(arr[len(arr)//2:])  # O(n) concat!
```

Reality: String/list concatenation is O(n), not O(1).

Exercise 2: Short Answer (15 min)

Instructions: Answer the following questions in complete sentences. Write in `exercise2.py` as a multi-line comment, then commit.

```
"""
Lecture 17 Exercise 2: Master Theorem Applications

1. Apply the Master Theorem to T(n) = 3T(n/2) + O(n).

   a) What are a, b, and f(n)?
   b) What is log_b a?
   c) Which case applies?
   d) What is the final time complexity?

   Answer:

2. Consider T(n) = T(n/4) + T(3n/4) + O(n).

   a) Why doesn't the standard Master Theorem apply?
   b) What method would you use instead?
   c) What do you expect the complexity to be? (Intuition is OK)

   Answer:

3. Explain why the following code is inefficient and how to fix it:

   def slow_sum(arr):
```

```
        if len(arr) == 0:
            return 0
        return arr[0] + slow_sum(arr[1:])  # arr[1:] creates copy

    Answer:

"""
```

Commit: `git add exercise2.py && git commit -m "Lecture 17 Exercise 2: Master Theorem short answer"`

Exercise 2 Sample Solutions

1. T(n) = 3T(n/2) + O(n)

a) a = 3, b = 2, f(n) = Θ(n)

b) log_b a = $\log_2 3 \approx 1.585$

c) Case 1 applies: f(n) = O(n^1) where 1 < 1.585

d) T(n) = Θ(n^($\log_2 3$)) ≈ **Θ(n^1.585)**

2. T(n) = T(n/4) + T(3n/4) + O(n)

a) Master Theorem requires equal split sizes. Here we have n/4 and 3n/4 (unequal).

b) Use recursion tree method or substitution method.

c) Expected complexity: O(n log n) — work is O(n) per level, height is log_{4/3} n ≈ O(log n).

3. Inefficient recursion:

Problem: arr[1:] creates a new array (O(n) copy) at each recursive call. With n calls, total time is $O(n^2)$ instead of expected O(n).

Fix: Use indices instead of slicing:

```
def fast_sum(arr, i=0):
    if i >= len(arr):
        return 0
    return arr[i] + fast_sum(arr, i + 1)
```

Or better: use iteration for linear recursion.

Complexity Checkpoints

Can you:

✓ **Write a recurrence** from code structure?

- Count recursive calls → a
- Find input size reduction → b
- Identify work at this level → f(n)

✓ **Apply the Master Theorem**?

- Compute log_b a
- Compare f(n) to n^(log_b a)
- Select correct case (1, 2, or 3)

✓ **Recognize when Master Theorem doesn't apply**?

- Unequal splits
- Non-polynomial f(n)
- Linear recursion (T(n-1) instead of T(n/b))

✓ **Identify Python-specific costs**?

- Array slicing creates copies
- Recursion depth limit
- String/list concatenation is O(n)

Summary

Key takeaways

Recurrences express recursive algorithm runtime:

- Base case: $T(1) = O(1)$
- Recursive case: $T(n) = a \cdot T(n/b) + f(n)$

Master Theorem (three cases):

1. **Leaves dominate:** $f(n) < n^{(\log_b a)} \rightarrow T(n) = \Theta(n^{(\log_b a)})$
2. **Balanced:** $f(n) = \Theta(n^{(\log_b a)}) \rightarrow T(n) = \Theta(n^{(\log_b a)} \log n)$
3. **Root dominates:** $f(n) > n^{(\log_b a)} \rightarrow T(n) = \Theta(f(n))$

Common patterns:

- Binary search: $T(n) = T(n/2) + O(1) \rightarrow O(\log n)$
- Mergesort: $T(n) = 2T(n/2) + O(n) \rightarrow O(n \log n)$
- Naive matmul: $T(n) = 8T(n/2) + O(n^2) \rightarrow O(n^3)$

Python considerations:

- Recursion depth limit (~1000)
- Array slicing creates copies (O(n))
- Use indices for divide-and-conquer

Why it matters

Algorithm design:

- Predict performance before implementing
- Compare divide-and-conquer strategies
- Understand tradeoffs (recursion depth vs code clarity)

Interview preparation:

- Master Theorem is a standard tool
- Quickly analyze recursive solutions
- Communicate complexity reasoning

Production systems:

- Know when recursion is safe (shallow) vs risky (deep)
- Optimize by reducing constant factors
- Choose iterative when recursion hits limits

Next lecture preview

Lecture 18: Algorithm Families and Complexity Comparisons

- Greedy, divide-and-conquer, dynamic programming, backtracking
- When to use each paradigm
- Complexity classes: P, NP, NP-complete
- Practical problem classification

Homework:

- Practice Master Theorem on textbook problems
- Review all recurrences from previous lectures
- Think about recursive algorithms you've written

Exam 3 on March 31

- Covers Q3 material: graphs, shortest paths, greedy, recurrences
- Applied problem-solving focus
- Theory supports, but coding is emphasized